

Szánkó osztály

Készíts **Szánkó** osztályt a következő UML diagram alapján:

Szanko
- letszam: int - allapot: int - maxTeherBiras: int
+ UresE {get;} : bool + Letszam { get; set;} : int + Allapot { get; set;} : int + Terheles {get; set;} : int + MaxTeherBiras {get; private set;} + Szanko() + Szanko(int maxTeherBiras) + Log() : string + FelUIValaki(int suly): bool + Csuszas() + Javitas(int ertek)

- Az **UresE** tulajdonság igaz, ha a létszám 0, különben hamis.
- A szánkón ülők létszáma csak 0 és 3 között lehet.
- Az állapot csak 0 és 100 közötti érték lehet. Amennyiben 0-nál kisebb értéket próbál valaki beállítani, akkor az érték 0 lesz. Ha valaki 100 feletti értéket próbál beállítani, akkor az érték 100 lesz.
- A **MaxTeherBiras** csak 0 és 200 közötti érték lehet. Amennyiben 0-nál kisebb értéket próbál valaki beállítani, akkor az érték 0 lesz. Ha valaki 200 feletti értéket próbál beállítani, akkor az érték 200 lesz.
- A paraméter nélküli konstruktor a létszámot 0-ra állítja, és véletlenszerűen állítja be az állapotot 50 és 100 közötti értékre. A terhelés 0, a maximális teherbírás pedig 100 és 200 közötti véletlen szám.
- A paraméteres konstruktor a paraméter alapján állítja be a maximális teherbírást. A többi adat ugyanúgy kap kezdőértéket, mint a paraméter nélküli konstruktornál.
- A **Log()** a szánkó terhelését és állapotát adja vissza. (Ld. minta)
- Csak akkor ülhet fel valaki a szánkóra, ha még nem ülnek rajta hárman. A **FelUIValaki()** metódus a felülés sikerességét adja vissza. Ha sikerül felülni, akkor a létszámot növelni kell, és a terhelés is megnő a paraméterül kapott súllyal. Ha így a terhelés túllépi a lehetséges maximális teherbírást, akkor az állapot 0 lesz.
- A **Csuszas()** csak akkor lehetséges, ha a szánkó nem üres. Ilyenkor az állapot a következő képlet alapján csökken:

$$\text{Terheles} * 3 / \text{maxTeherBiras}$$

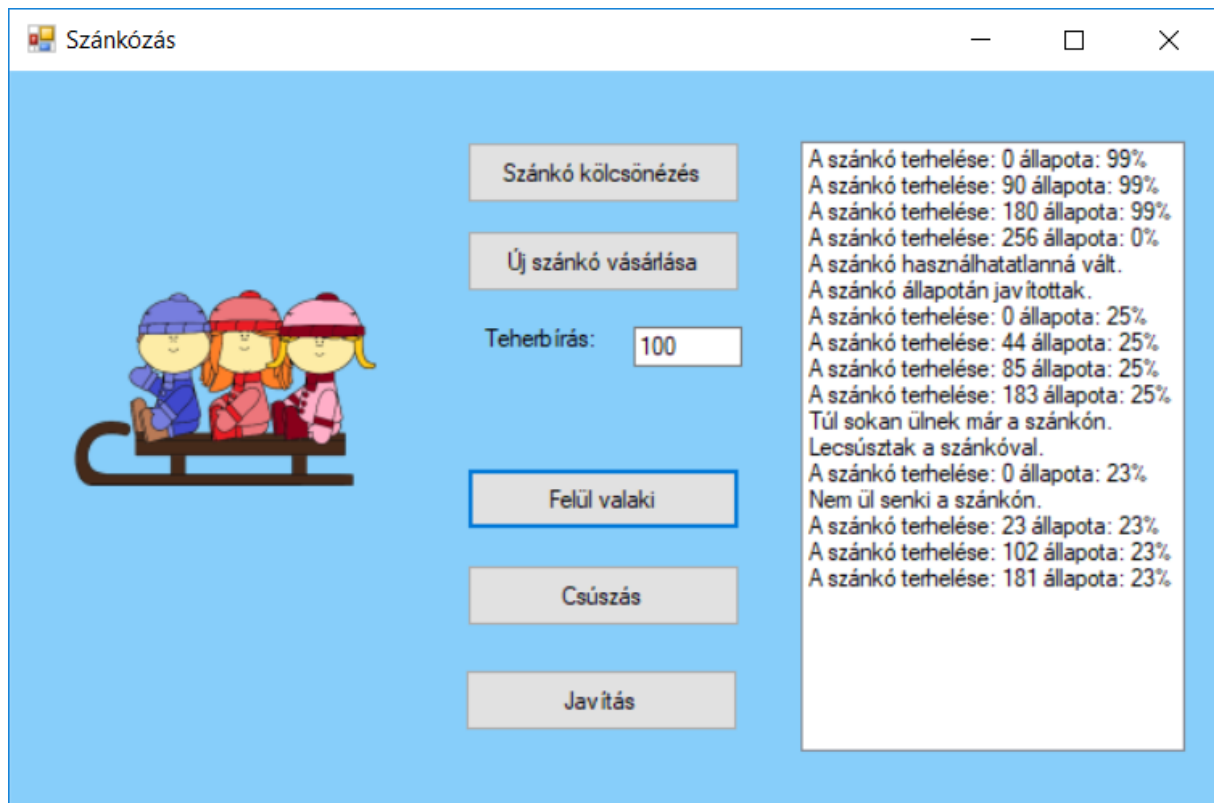
A képlet egész osztással számol. Csúszás után az emberek leszállnak a szánkóról, ezért a létszám és a terhelés 0-ra áll vissza.

- A **Javitas ()** a paraméterül kapott értékkel növeli az állapotot. Mielőtt javítani kezdenék a szánkót, az embereknek le kell szállnia róla. Természetesen ilyenkor terhelés sincs rajta.

Tesztprogram

Készíts tesztprogramot az osztály kipróbálásához!

A megvalósításhoz nem kötelező a mintát követni, ez csak egy ajánlás, de minden felsorolt funkciót meg kell valósítani valamilyen módon! A tesztprogramot meg lehet írni konzolban is. Ebben az esetben a logolás egy txt fájlban is történjen meg!



- Lehessen szánkót kölcsönözni, paraméterek megadása nélkül! A kölcsönzés után jelenítsd meg a szánkóhoz tartozó információkat!
- Lehessen szánkót vásárolni, ahol meg kell adni a maximális teherbírás paraméterét! A vásárlás után jelenítsd meg a szánkóhoz tartozó információkat!
- Amikor a szánkóra felül valaki, véletlenszerűen generálni kell a súlyát, mely 10 és 100 közötti érték legyen. Ha a szánkó teherbírása így túllépi a megengedettet, akkor a szánkó összetörik, ki kell írni, hogy a szánkó használhatatlanná vált. Természetesen a szánkóra csak akkor ülhet fel új ember, ha még nem ülnek rajta hárman. Minden felülési kísérlet után jelenjen meg a szánkó aktuális állapota! Természetesen arról is információt kell kiírni, ha nem lehet már felülni rá, mert már hárman ülnek rajta.
- Lecsúszni a szánkóval csak akkor lehet, ha ül rajta valaki. Jelenjen meg a logban, ha üres szánkóval szeretne valaki csúszni, vagy ha sikeres a csúszás, akkor a csúszás utáni állapot.
- Javításkor a szánkó állapota egy 10 és 30 közötti véletlen értékkel nő. Jelenjen meg a logban, hogy a szánkó állapotán javítottak, valamint a szánkóhoz tartozó információk is!

Jó munkát!